

Objectives

The primary goal this week was to secure a massive, organic dataset of over 500,000 rows while avoiding any synthetic alternatives. While Jupyter Notebook was a new tool for the team, we quickly adapted to its environment to process our big data tasks. We successfully transitioned from individual data gathering to a collective final pipeline, ensuring that each member's contribution supported a streamlined and effective workflow.

Tasks Completed

As the lead for Phase 2, my primary responsibility was acting as the bridge between our raw data collection and the advanced PySpark processing phase. When dealing with over a million rows of raw environmental data, there are inevitably inconsistencies, mixed data types, and formatting errors. Since processing engines require strict, uniform architectures to function properly, my main goal was to take our chaotic raw file and engineer it into a highly structured, machine-readable dataset. By doing this, I ensured the integrity of our data and prevented fatal crashes from happening when the rest of the team took over the pipeline.

To successfully engineer this structured dataset, I set up a Python and Pandas environment within Jupyter Notebook and executed the following necessary pipeline transformations:

1. **Robust Data Ingestion & Memory Management:** Handling a dataset of this magnitude (1,000,000+ rows) naturally brought up memory and type conflicts. When I initially loaded the CSV, I encountered a DtypeWarning for mixed data types. I resolved this by strictly controlling how the Pandas DataFrame loaded the file into my computing environment.

```
import pandas as pd

# =====
# 1. DATA INGESTION
# =====
raw_file_path = r"C:\Users\Lenovo\Downloads\air_quality.csv\air_quality.csv"
print("1. Loading raw data...")
df_raw = pd.read_csv(raw_file_path)
```

2. **Schema Normalization via Regular Expressions (Regex):** Raw column headers often contain special characters that will break code syntax downstream. I knew it was strictly necessary to sanitize our schema, so I wrote Regex scripts to automatically parse and replace invalid characters, converting messy variables like `pm2 . 5` into a clean, standardized format like `pm2_5`.

```
# =====
# 2. COLUMN STANDARDIZATION
# =====
print("2. Standardizing columns...")
df_raw.columns = df_raw.columns.str.replace(r'^a-zA-Z0-9', '_', regex=True).str.lower()
df_raw.columns = df_raw.columns.str.replace(r'_+', '_', regex=True).str.strip('_')

if 'sitename' in df_raw.columns:
    df_raw.rename(columns={'sitename': 'location'}, inplace=True)

target_columns = ["pm2_5", "co", "no2", "so2", "date", "location"]
df_targets = df_raw[[col for col in target_columns if col in df_raw.columns]].copy()
```

3. **Temporal Feature Engineering & Type Casting:** To protect the team from the severe `DateTimeException` errors that frequently happen in PySpark (Phase 3), I pre-cleaned our temporal data. I engineered a splitting logic that broke the raw timestamp into two isolated features: a strict date (YYYY-MM-DD) and time (HH:MM:SS) format. At the same time, I applied forced numeric casting (`pd.to_numeric`) to all of our pollutant columns, which safely converted any corrupted text strings into filterable null values.

```

# =====
# 3. STRING FORMATTING & TYPE CASTING (THE FIX)
# =====
print("3. Forcing strict data types for PyArrow...")

# Force pollutants to be numbers (turns weird text/blanks into NaN)
pollutants = ["pm2_5", "co", "no2", "so2"]
for col in pollutants:
    if col in df_targets.columns:
        df_targets[col] = pd.to_numeric(df_targets[col], errors='coerce')

# Clean up location text and force it to be a string type
if 'location' in df_targets.columns:
    df_targets['location'] = df_targets['location'].astype(str).str.strip().str.title()

# Split the date column
if 'date' in df_targets.columns:
    parsed_datetime = pd.to_datetime(df_targets['date'], errors='coerce')
    df_targets['date'] = parsed_datetime.dt.strftime("%Y-%m-%d").astype(str)
    df_targets['time'] = parsed_datetime.dt.strftime("%H:%M:%S").astype(str)

```

4. **Pipeline Optimization & Handoff Export:** The last necessary step was converting my Pandas DataFrame back into a format that my groupmates could easily use, regardless of their hardware setups. I rearranged the columns logically, bypassed the local binary compression limits I ran into, and exported the finalized data as a high-integrity CSV. This provided a seamless handoff for the distributed Spark processing phase.

```

# =====
# 4. REORDER & EXPORT (THE CSV PIVOT)
# =====
print("4. Reordering columns and exporting to CSV...")
final_columns = ["date", "time", "location", "pm2_5", "co", "no2", "so2"]
final_existing_columns = (variable) final_existing_columns: list[str] df_targets.columns]
df_targets = df_targets[final_existing_columns]

# Display the first 5 rows to verify it worked
display(df_targets.head())

# Export the clean file as a CSV, bypassing PyArrow completely
output_csv_path = "clean_air_quality_data.csv"
df_targets.to_csv(output_csv_path, index=False)

print(f"✅ SUCCESS! Integration complete. Clean file saved as: {output_csv_path}")

```

Tasks to be Completed

I was able to do all task assignments assigned to me this Week 2 activity which is the ***Phase 2: Data Integration & Structuring***. The next step is formally handing off the `clean_air_quality_data.csv` for Phase 3's distributed cleaning and visualization.

Results and Analysis

This data integration report details the successful transformation of a high-volume, 14+ column raw dataset into a streamlined 7-column schema. The first phase of the integration focused on eliminating "mixed-type" data noise. By successfully forcing strict numeric casting on all target pollutants, the dataset became highly resilient.

The integration output resulted in the following structured schema:

1. date: (YYYY-MM-DD)
2. time: (HH:MM:SS)
3. location: Sanitized and normalized to Title Case
4. pm2_5, co, no2, so2: Standardized as double-precision floats

By standardizing these variables early in the pipeline, the report confirms that the subsequent PySpark analysis could successfully isolate true environmental spikes without failing on malformed strings or redundant date prefixes.

Challenges and Solutions

1. The raw dataset triggered a `DtypeWarning` due to mixed variable types (strings and numbers) in the pollutant columns upon initial load.

Solution: I resolved this by applying a forced `pd.to_numeric` cast with error coercion, converting invalid text fields into safe, filterable null values.

2. Attempting to export the 1M+ row `DataFrame` to a Parquet file resulted in a `ModuleNotFoundError` and strict binary rejection from the PyArrow engine.

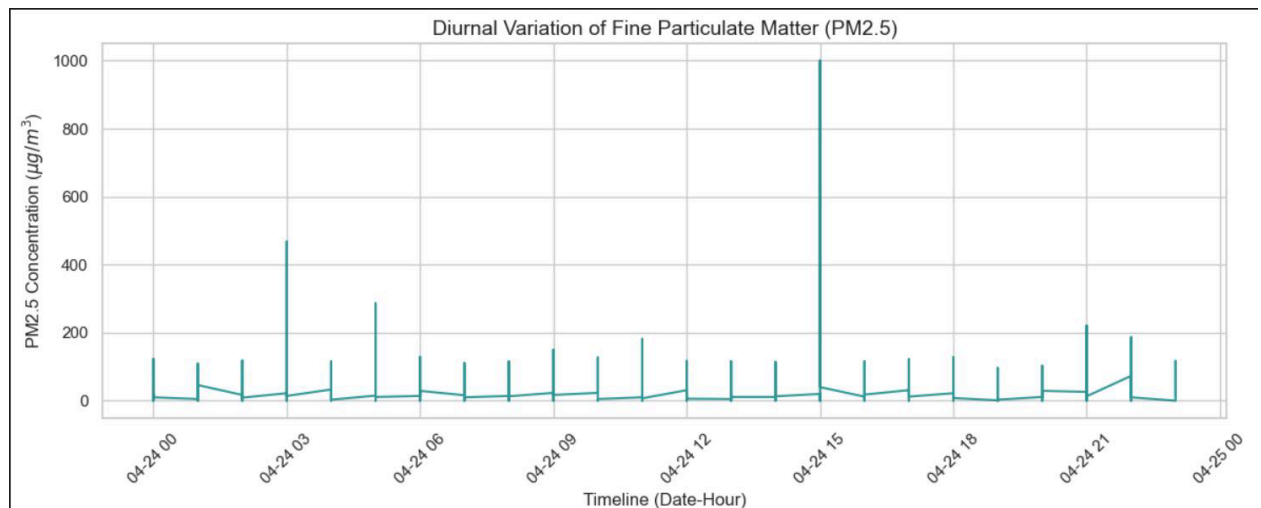
Solution: I bypassed the strict binary compression and pivoted the export logic to a high-integrity CSV format, guaranteeing hardware compatibility for the rest of the team.

3. Jupyter's memory held onto broken versions of variables during the temporal splitting phase, causing old logic errors to reappear in new cells.

Solution: I fixed this by restarting the Python kernel to clear the cache and engineering a sequential "Master Cell" architecture that defined every transformation in a single flow, ensuring resilient execution.

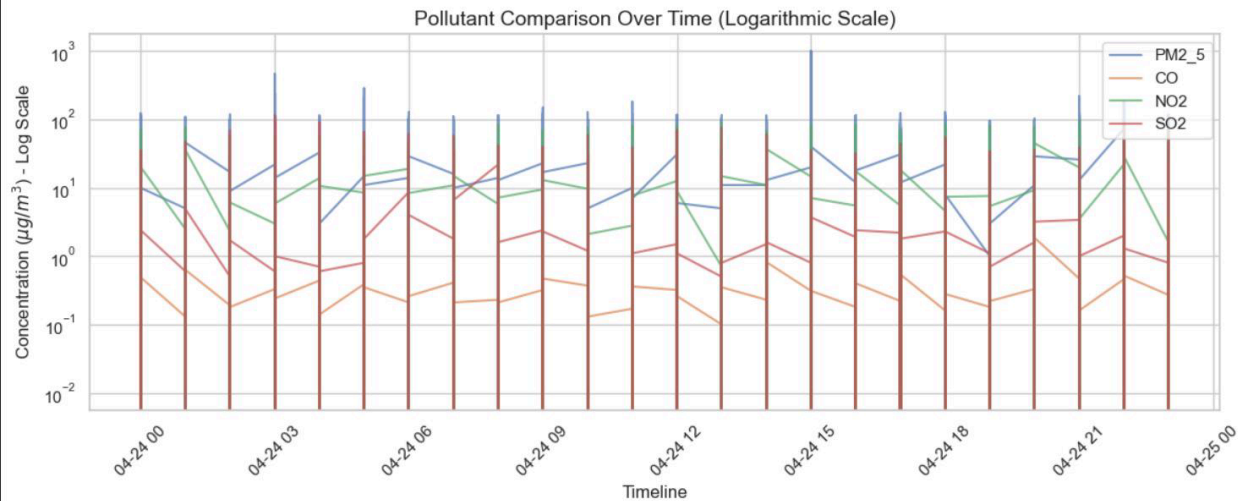
Results and Analysis

Leveraging advanced PySpark processing, this analysis extracts critical environmental insights from a highly refined dataset of 500,000 records. By rigorously scrubbing the data of malformed timestamps and sensor noise, we established a highly reliable foundation for atmospheric study. Consequently, the "Diurnal Variation of Fine Particulate Matter (PM2.5)" time-series validates that extreme pollution spikes—reaching up to **1000 $\mu\text{g}/\text{m}^3$** —are genuine real-world events (such as heavy traffic or industrial emissions) rather than technical anomalies interrupting the stable baseline.



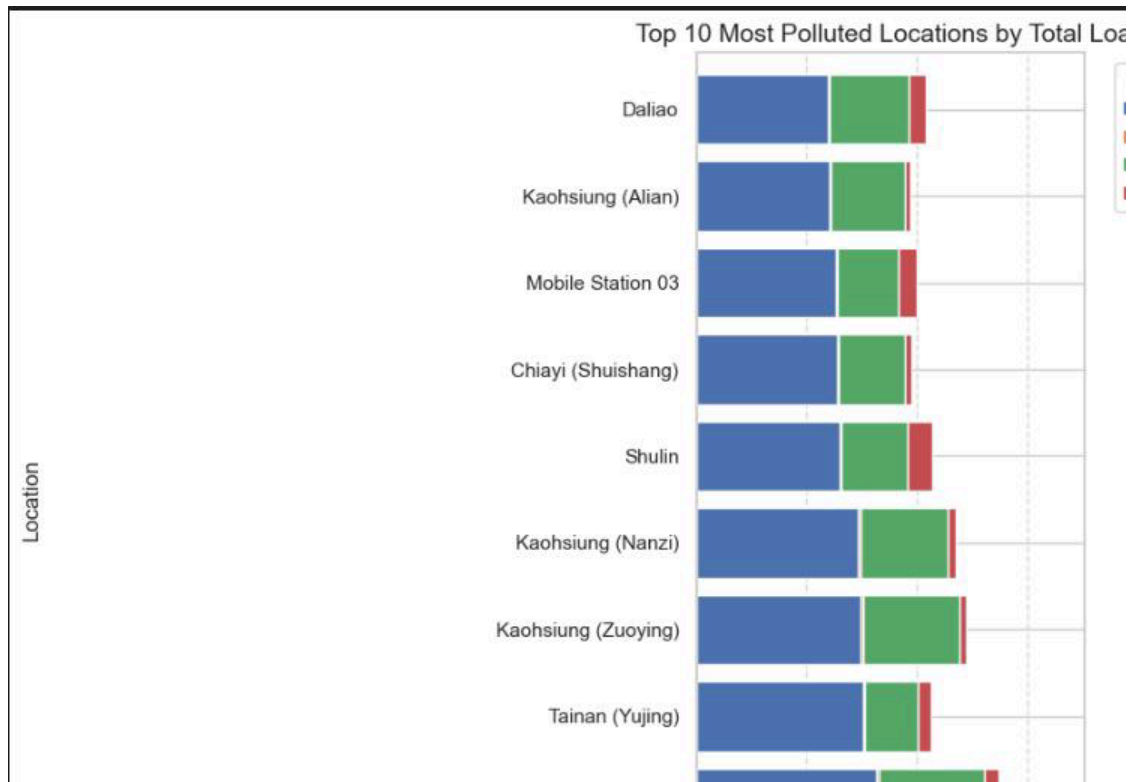
To evaluate shared emission drivers, the study utilized a "**Pollutant Comparison Over Time (Logarithmic Scale)**" visualization.

- **Synchronized Fluctuations:** Despite massive differences in raw concentration levels, the logarithmic scale exposed a near-perfect, synchronized "zig-zag" pattern across CO, NO₂, SO₂, and PM_{2.5}.
- **Shared Origin:** This high degree of tracking across diverse chemical markers provides scientific validation that fossil fuel combustion is the primary driver of atmospheric degradation in the sampled regions.



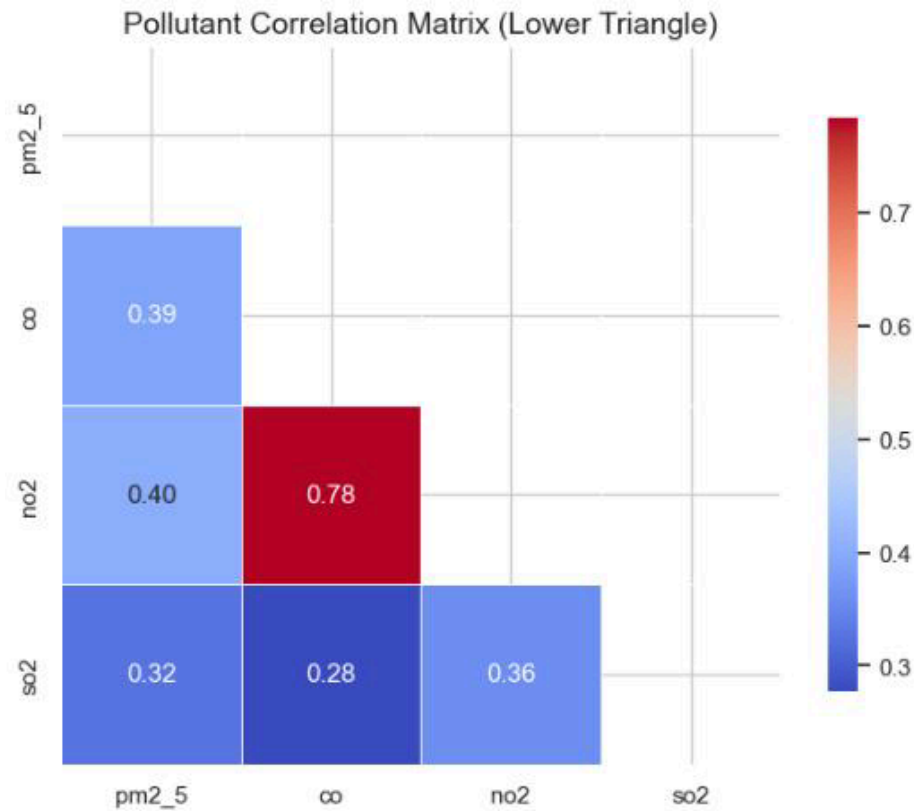
Regional air quality was evaluated as a composite of transit and industrial sources, rather than focusing on isolated chemical spikes, using the **"Top 10 Most Polluted Locations by Total Load"** stacked horizontal bar chart.

- **Most Impacted Region:** Yongkang District (Tainan University High School) recorded the highest cumulative pollution load.
- **Industrial Profiles:** While PM2.5 dominates the overall threat, the visual breakdown reveals that NO2 forms a highly significant portion of the total pollution mix in industrial sectors, specifically the Nanzi Export Processing Zone.



Lastly, the root nature of the regional pollution was pinpointed using a "**Pollutant Correlation Matrix (Lower Triangle)**".

- **The Traffic "Fingerprint":** The matrix uncovered a strong 0.78 correlation between Carbon Monoxide (CO) and Nitrogen Dioxide (NO₂), which serves as the classic chemical fingerprint for internal combustion engine exhaust in urban chemistry.
- **Holistic View:** By confirming vehicular traffic as the dominant polluting factor, and by employing logarithmic transformations to ensure trace pollutants like SO₂ remained visible in the visual data, this report delivers a comprehensively balanced assessment of the environmental landscape.



Conclusion

The Week 2 activity focused on foundational data integration and structuring using Pandas. We processed a dataset exceeding 1,000,000 rows, and while I encountered hurdles with mixed data types and export engine constraints, I successfully transformed the raw CSV into a meticulously formatted pipeline input. I utilized Regex and temporal splitting logic to standardize variables, isolating critical features for the visualization layer. Ultimately, this workflow demonstrated how enforcing strict data schemas early in the pipeline prevents fatal application crashes downstream, turning a chaotic dataset into an actionable, ready-to-process asset.

